

Prova Finale di Reti Logiche

Prof. Gianluca Palermo - Anno 2022-2023

Luca Pagano - Codice Persona: 10769185



POLITECNICO
MILANO 1863

Introduzione

Il modulo da implementare ha due ingressi primari da 1 bit (W e START), 5 uscite primarie, di cui 4 da 8 bit (Z0, Z1, Z2, Z3) ed un segnale DONE. Inoltre, il modulo ha un segnale di clock CLK, unico per tutto il sistema e un segnale di RESET.

Il segnale di START viene portato ad 1 per un periodo che varia dai 2 ai 18 cicli di clock, durante il quale il componente riceve in input, sul segnale W, una sequenza di bit rappresentanti il canale e l'indirizzo di memoria ordinati dal MSB al LSB. I primi due bit letti rappresenteranno l'indirizzo dell'uscita Z_i, e i rimanenti bit serviranno per calcolare l'indirizzo di 16 bit. Con esso potrà prelevare dalla memoria il messaggio da mostrare alla corrispondente uscita; qualora i bit letti non dovessero essere sufficienti, l'indirizzo letto fino a quel momento verrà esteso con degli zeri nel seguente modo:

(N = 7)	1010111	-> 0000000001010111
(N = 16)	1110000001010111	-> 1110000001010111
(N = 0)	0000000000000000	-> 0000000000000000

Le uscite Z0, Z1, Z2 e Z3 sono inizialmente poste a 0. I valori rimangono inalterati eccetto il canale sul quale viene mandato il messaggio letto in memoria. Essi sono visibili solo quando il valore di DONE è 1, in caso contrario in uscita avremo solo zeri. Il periodo di computazione del modulo, ovvero il tempo che intercorre fra la discesa del segnale START e la salita del segnale DONE, non deve eccedere i 20 cicli di clock.

L'interfaccia del componente è descritta nel seguente modo:

```
entity project_reti_logiche is
  port (
    i_clk   : in std_logic;
    i_rst   : in std_logic;
    i_start : in std_logic;
    i_w     : in std_logic;
    o_z0    : out std_logic_vector(7 downto 0);
    o_z1    : out std_logic_vector(7 downto 0);
    o_z2    : out std_logic_vector(7 downto 0);
    o_z3    : out std_logic_vector(7 downto 0);
    o_done  : out std_logic;
    o_mem_addr : out std_logic_vector(15 downto 0);
    i_mem_data : in std_logic_vector(7 downto 0);
    o_mem_we  : out std_logic;
    o_mem_en  : out std_logic;
  );
end project_reti_logiche;
```

Dove:

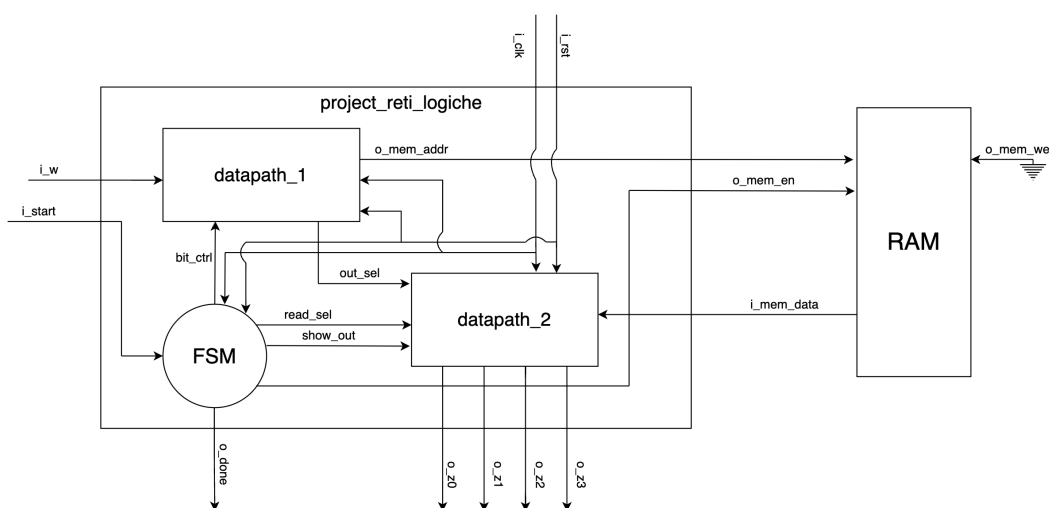
- ***i_clk***: segnale di clock in ingresso generato dal Test Bench;
- ***i_rst***: è il segnale di RESET che inizializza la macchina pronta per ricevere il primo segnale di START;
- ***i_start***: è il segnale di START generato dal Test Bench;
- ***i_w***: è il segnale W precedentemente descritto e generato dal Test Bench;
- ***i_mem_data***: è il segnale (vettore) che arriva dalla memoria in seguito ad una richiesta di

lettura;

- **o_mem_addr**: è il segnale (vettore) di uscita che manda l'indirizzo alla memoria;
- **o_z0, o_z1, o_z2, o_z3**: sono i quattro canali di uscita;
- **o_done**: è il segnale di uscita che comunica la fine dell'elaborazione;
- **o_mem_we**: è il segnale di WRITE ENABLE da dover mandare alla memoria (=1) per poter
scriverci. Per leggere da memoria esso deve essere 0.
- **o_mem_en**: è il segnale di ENABLE da dover mandare alla memoria per poter
comunicare (sia
in lettura che in scrittura);

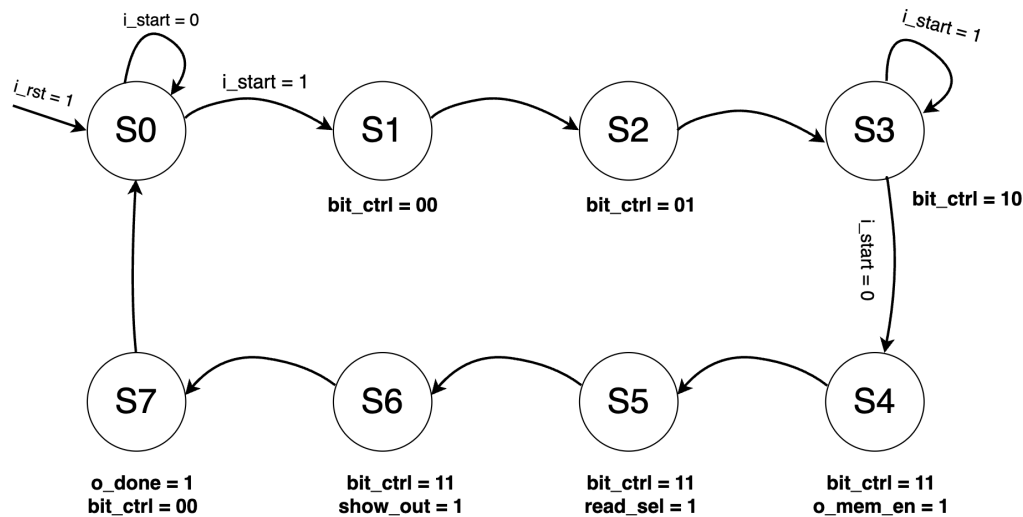
Progettazione

Il componente `project_reti_logiche` viene realizzato ad alto livello nel seguente modo, con due sotto-componenti `datapath_1` e `datapath_2` adibiti rispettivamente alla lettura dell'input e alla gestione delle uscite.



Queste entità sono controllate da una macchina a stati, così da poter gestire le diverse fasi di elaborazione del componente. I segnali di controllo, prodotti dalla macchina a stati, quando non indicati esplicitamente sono posti di default a zero; inoltre, come scelta progettuale, ho adottato una Macchina di Moore così da poter

separare nei processi la gestione delle uscite da quella di transizione da uno stato ad un altro.



Descrizione degli Stati

Stato	Descrizione
S0	Esso rappresenta lo stato iniziale, invocato ogni qual volta il segnale di reset viene portato ad 1 o quando viene completata la fase di lettura del bit stream, dopo aver mostrato i valori delle uscite. Tutti i valori vengono inizializzati a zero
S1	Il componente riceve il segnale di <i>i_start</i> ad 1 e inserisce il primo valore letto di <i>i_w</i> nel bit più significativo di <i>out_sel</i>
S2	Il componente dopo un colpo di clock legge il secondo valore di <i>i_w</i> e lo inserisce nel bit meno significativo di <i>out_sel</i> . Dopo questo stato sapremo in quale uscita mostrare successivamente il contenuto di <i>i_mem_data</i>
S3	Il segnale <i>i_w</i> viene letto finché il segnale <i>i_start</i> rimane ad 1 e salvato sul registro <i>z_address</i> ; il contenuto precedente, inizialmente inizializzato a 0, viene di volta in volta shiftato a sinistra di un bit e il valore dell'input inserito nel bit meno significativo
S4	Il segnale di input è stato letto dal componente e il segnale di start è stato riportato a zero; viene portato a valor logico alto il segnale di <i>o_mem_en</i> , consentendo la lettura in memoria del contenuto dell'indirizzo <i>o_mem_addr</i>

Stato	Descrizione
S5	Viene posto <i>read_sel</i> = 1 così da poter inserire nel registro corrispondente all'uscita indicata in <i>out_sel</i> il contenuto letto dalla memoria
S6	Portiamo il valore di <i>show_out</i> ad 1 così da poter mostrare al successivo ciclo di clock i valori delle uscite
S7	Mostriamo finalmente il contenuto delle uscite contemporaneamente portando il segnale di <i>o_done</i> = 1 e ritorniamo allo stato iniziale

Descrizione segnali

Nome	Descrizione
bit_ctrl	È il segnale che gestisce la lettura del bitstream in input in <i>datapath_1</i> . Esso assume un significato diverso a seconda dei valori: quando vale "00" salvo il valore di <i>i_w</i> nel MSB di <i>out_sel</i> , per "01": salvo il valore di <i>i_w</i> invece nel LSB. Per "10" si ha la lettura dell'indirizzo da salvare ed infine per "11" lascio invariati i vari registri
show_out	Esso indica a <i>datapath_2</i> se mostrare il contenuto dei registri <i>s_z</i> oppure se mostrare '0'
read_sel	Se il segnale è alto allora viene scritto nel registro i-esimo, indicato in <i>out_sel</i> , il contenuto di <i>i_mem_data</i>
out_sel	Segnale passato a <i>datapath_2</i> che indica i bit di intestazione riguardanti l'uscita sul quale mostrare il contenuto di <i>i_mem_data</i>
z_selector	Registro dove salvo inizialmente il valore da passare ad <i>out_sel</i>
z_address	Registro sul quale svolgo le operazioni di shift logico a sinistra durante la lettura dell'indirizzo da cui leggere in memoria
tmp_i_w	In esso salvo il valore di <i>i_w</i> per sincronizzare correttamente il circuito.
s_z0, s_z1, s_z2, s_z3	Rappresentano i registri a 8 bit dove salvo il contenuto da mostrare alle corrispettive uscite <i>o_z0</i> , <i>o_z1</i> , <i>o_z2</i> , <i>o_z3</i> quando porto il segnale di DONE ad 1

Report di Sintesi

La macchina progettata risponde efficacemente alla specifica, utilizzando meno del 3% del periodo di clock a disposizione dei test-bench assegnati. Inoltre, come indicato nel *report_utilization*, la sintesi non ha necessitato l'inferenza di alcun latch.

Report Timing

```

Slack (MET) :          97.435ns  (required time - arrival time)
  Source:            FSM_sequential_cur_state_reg[0]/C
                    (rising edge-triggered cell FDCE clocked by clock  {rise@0.000ns fall@50.000ns period=100.000ns})
  Destination:       DATAPATH2/s_z2_reg[0]/CE
                    (rising edge-triggered cell FDCE clocked by clock  {rise@0.000ns fall@50.000ns period=100.000ns})
  Path Group:         clock
  Path Type:          Setup (Max at Slow Process Corner)
  Requirement:        100.000ns  (clock rise@100.000ns - clock rise@0.000ns)
  Data Path Delay:    2.183ns  (logic 0.779ns (35.685%)  route 1.404ns (64.315%))
  Logic Levels:       1  (LUT5=1)
  Clock Path Skew:    -0.145ns  (DCD - SCD + CPR)
    Destination Clock Delay (DCD):  2.100ns = ( 102.100 - 100.000 )
    Source Clock Delay (SCD):        2.424ns
    Clock Pessimism Removal (CPR):    0.178ns
  Clock Uncertainty:  0.035ns  ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
    Total System Jitter (TSJ):        0.071ns
    Total Input Jitter (TIJ):          0.000ns
    Discrete Jitter (DJ):              0.000ns
    Phase Error (PE):                 0.000ns

```

Report Utilization

1. Slice Logic

Site Type	Used	Fixed	Available	Util%
Slice LUTs*	34	0	134600	0.03
LUT as Logic	34	0	134600	0.03
LUT as Memory	0	0	46200	0.00
Slice Registers	86	0	269200	0.03
Register as Flip Flop	86	0	269200	0.03
Register as Latch	0	0	269200	0.00
F7 Muxes	0	0	67300	0.00
F8 Muxes	0	0	33650	0.00

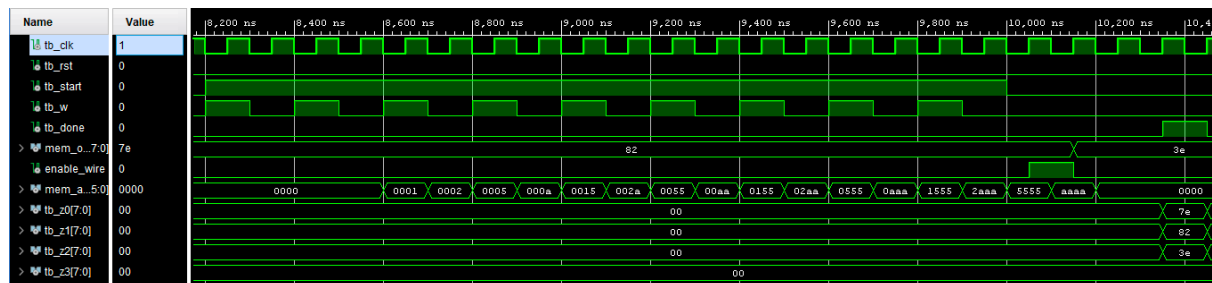
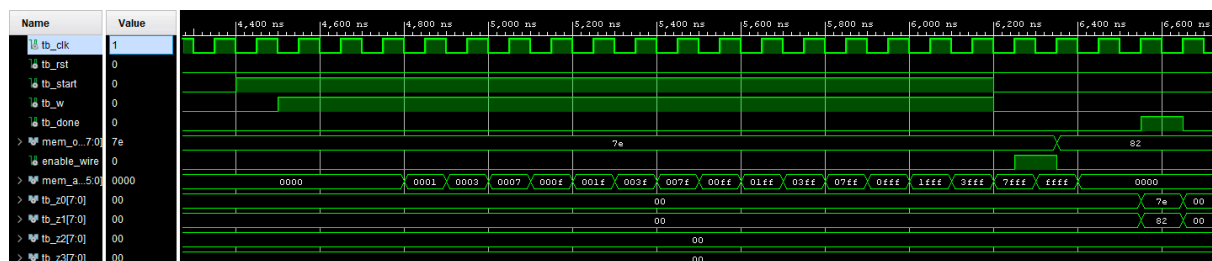
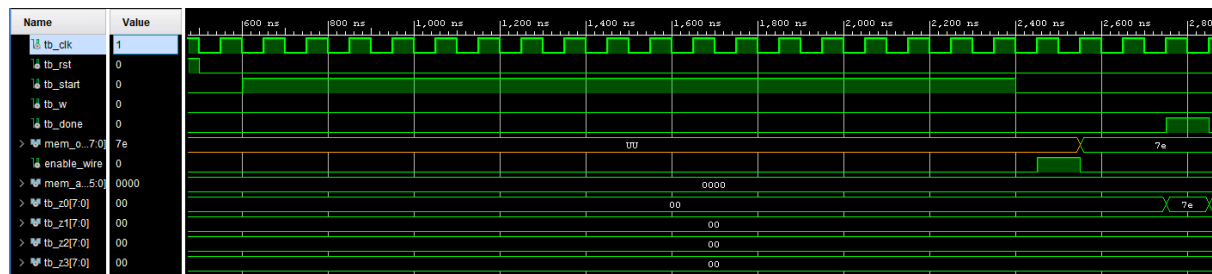
Testing

Il componente è stato testato in **Behavioural** e in **Post-Sintesi Funzionale** in **Vivado 2018.3.1** ottenendo in tutti i casi l'esito atteso.

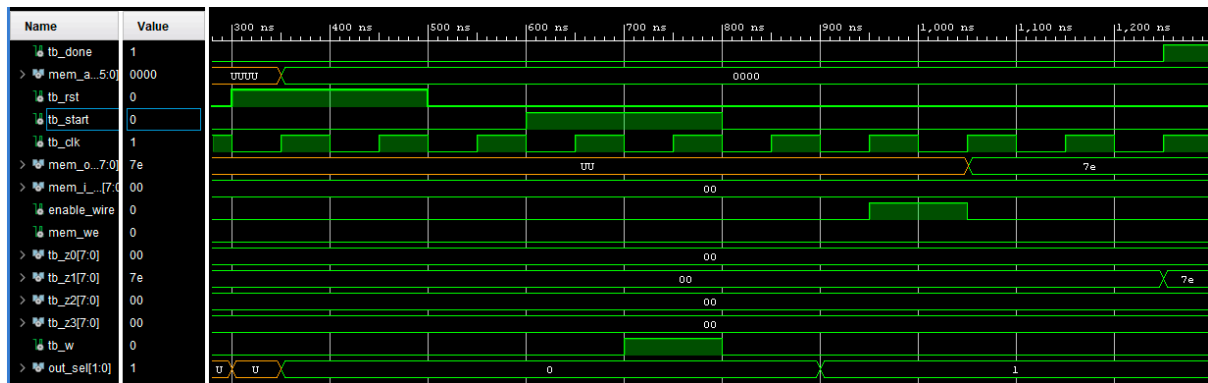
Oltre i test forniti dal docente e dei test generati casualmente, ho esaminato dei casi limite, specifici per il mio modulo, che ho scritto manualmente per verificare il funzionamento del componente.

Casi particolari di i_w

In esso controllo se il componente è in grado di leggere casi limite come quello in cui l'indirizzo è composto da solamente '0', solamente '1', e '1' e '0' alternati; il segnale di START viene quindi tenuto alto per 18 cicli di clock.



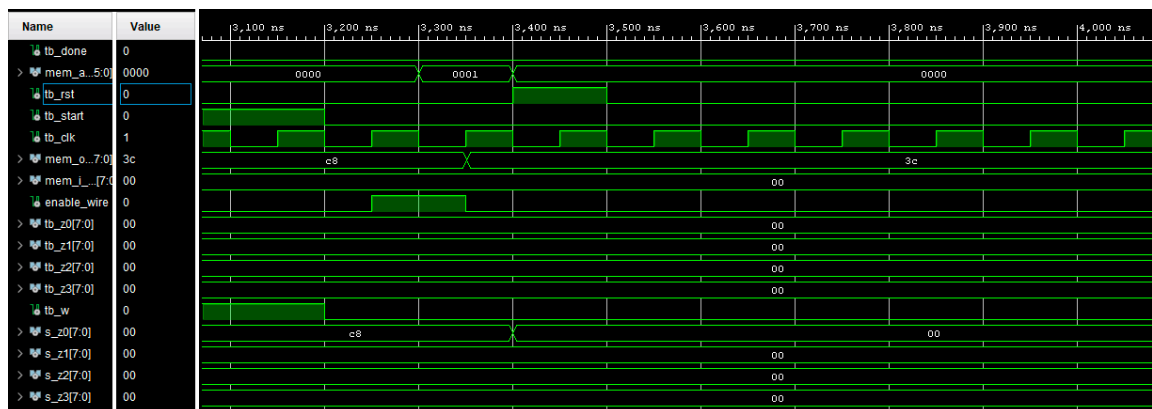
Infine ho controllato il caso in cui il segnale di START rimane alto per il numero minimo di clock necessari a soddisfare i requisiti della specifica, ovvero 2.



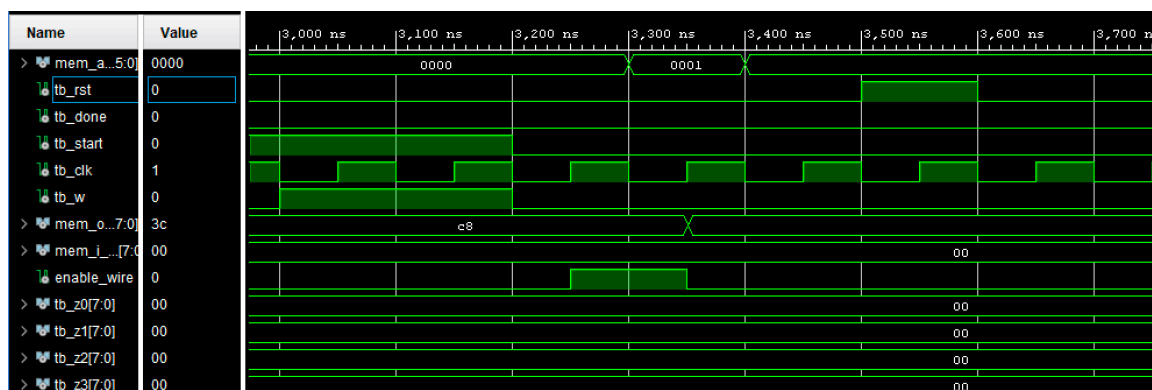
Casi limite Reset

Ho voluto testare il comportamento del modulo di fronte ad un segnale di reset alto in diverse fasi dell'elaborazione.

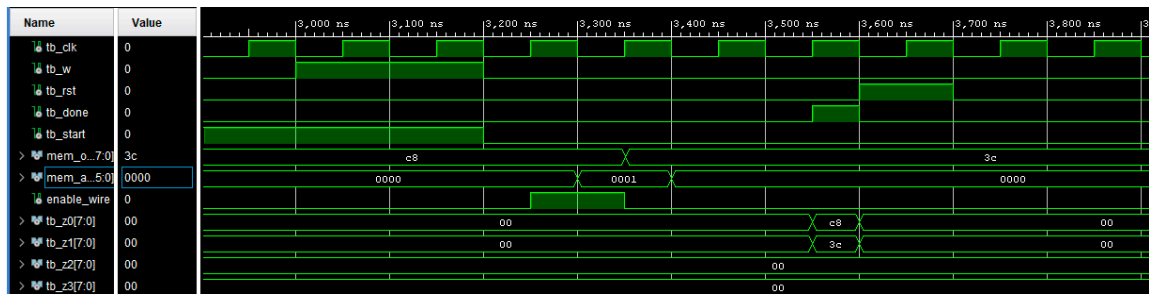
1. Reset durante l'elaborazione del componente



2. Reset sul tick del done



3. Reset subito dopo il tick del done



Conclusioni

Sono emerse alcune difficoltà durante la progettazione di questo modulo, in particolare in due momenti:

- Inizialmente il comportamento ottenuto in Behavioural non coincideva con quello riportato in Post-sintesi Funzionale. Dopo aver analizzato l'output, mi sono reso conto che esso era identico a quello ottenuto in comportamentale in assenza di *tmp_i_w*, facendomi intuire che in sintesi veniva ignorato. Pertanto, dopo aver individuato l'origine del problema, sono riuscito a far coincidere l'output delle due tipologie di simulazioni, portando il segnale *tmp_i_w* in un processo a parte, forzando la creazione di un registro.
- Il secondo riguardava il comportamento del componente nel caso in cui alzavo il segnale di RESET subito dopo il DONE; le uscite venivano reinizializzate a zero dopo un ciclo di clock quando la commutazione in realtà doveva essere istantanea. L'errore derivava dal fatto che il RESET azzerava i registri *s_z* ma non le uscite *o_z*, le quali venivano aggiornate solo sul fronte di salita del clock.

È anche grazie al superamento di questi ostacoli che ho potuto acquisire competenze nella progettazione di componenti logici, utilizzando un software adoperato realmente in ambito lavorativo, potendo così mettere in pratica le conoscenze teoriche apprese durante il corso di Reti Logiche.